

Homework 02 – Sample solution

QMIR 2026-fall: practising the reproducible workflow

Tristan Munro

September 20, 2026

Table of contents

What a complete submission looks like	1
Exercise 1: clone and open the project	2
Exercise 2: add metadata (YAML)	2
Exercise 3: writing in markdown	3
Exercise 4: image, table and cross-references	3
The image	3
The table	4
A first code chunk	5
Exercise 5: reflection	5
The git history you should end up with	6
Common misconceptions	7
Session info and execution time	7

What a complete submission looks like

This week has no right answer in the usual sense, because what is being practised is the workflow rather than an analysis. A complete submission is a repository `hw-02-
<username>` inside the course organisation that contains three things: the Quarto

source file, the rendered PDF, and a commit history that shows the document being built in steps.

The document below is one version of that submission, written out in full. Yours should differ in wording, in the image you chose and in what you found difficult. It should not differ in structure: every element the homework asks for is present, labelled, and referred to from the text.

Exercise 1: clone and open the project

Cloning copies the repository from GitHub to your machine and keeps the link between the two. From the terminal, inside the folder where your course material lives:

```
git clone https://github.com/qmir-2026-fall/hw-02-your-username.git
cd hw-02-your-username
```

Then open that folder in Positron with **File > Open Folder**. Two things confirm the setup worked. The Source Control pane shows the repository name instead of offering to initialise a new repository, and the terminal prompt sits inside the cloned folder, which you can check with `pwd`.

If you created the repository from the template link on the schedule page, the starting state is a repository with `README.md` and nothing else. That is expected for this week. You create `hw02.qmd` yourself, which is where Exercise 2 begins.

Exercise 2: add metadata (YAML)

The YAML header is the block at the very top of the file, fenced by three hyphens on their own line above and below. It is metadata: it tells Quarto what to build and what to put on the title page.

```
---
title: "Homework 02: Practising the reproducible workflow"
author: "Your Name"
date: today
format:
  pdf:
    toc: true
---
```

Each key does one job. `title` and `author` print on the first page, `date: today` fills in the render date so the document dates itself, `format: pdf` selects the output, and `toc: true` builds the table of contents from your headings. Indentation carries meaning in YAML, so `toc: true` has to sit two spaces further in than `pdf:`, which in turn sits two spaces further in than `format:`.

Render at this point, before writing any content. A header that renders is a header you can trust, and an error here is much easier to read when the document is still four lines long.

Exercise 3: writing in markdown

Quarto is an open-source publishing system that lets prose, code and output live in one document. You write in a plain text file with the extension `.qmd`, and Quarto *renders* it into a PDF, an HTML page or a Word file. Because the numbers in the text come from the code in the same file, the document cannot quietly fall out of step with the analysis behind it.

Git is a version control system. It records the state of your files each time you commit, so the history of a project becomes a sequence of readable steps rather than a folder of files called `final`, `final_v2` and `final_really`. Git runs locally, on your own machine, and needs no internet connection to work.

GitHub is a website that hosts Git repositories. It is where a local repository goes when you *push* it, and it is what makes the work visible to somebody else, which is how homework is submitted in this course.

The three fit together like this:

- Quarto turns source into output.
- Git records how the source changed.
- GitHub publishes both to a place other people can reach.

Exercise 4: image, table and cross-references

The image

An image is included with Markdown image syntax, not with R code. The text in square brackets is the caption, the path in round brackets points at the file, and the curly braces carry the label and any options.

```
![Caption text.] (images/quarto-logo.png){#fig-quarto-logo width=40%}
```



Figure 1: The Quarto logo. Quarto is the publishing system behind every document in this course, from the homework sheets to the lecture slides. Logo by Posit PBC, reproduced here for teaching.

Figure 1 shows the logo of the system that produced this document. Note what the reference does: the word “Figure” and the number are generated by Quarto, so inserting another figure earlier rennumbers everything correctly and the sentence stays true. Writing “the figure below” instead would break the moment anything moved.

Two details matter and are easy to miss. The label has to start with `fig-`, because that prefix is what tells Quarto the thing is a figure, and the image needs caption text, because a label with no caption is not a cross-reference at all. The caption itself should be self-sufficient: a reader who sees only the picture and its caption should understand what it is and why it is there.

The table

A table you type out by hand is written in Markdown, with the caption on the line directly below it. The caption line starts with a colon and ends with the label in curly braces.

Table 1: The six steps of the reproducible workflow practised in this homework, and what each one does. Staging is the step most often skipped, and a change that is not staged is not in the commit.

Step	Command or action	What it does
Clone	<code>git clone <url></code>	Copies the repository from GitHub to your machine
Edit	Positron editor	Changes the source file
Render	<code>quarto render</code> or Preview	Builds the PDF from the source
Stage	Source Control, plus button	Marks which changes belong in the next commit
Commit	<code>git commit -m "..."</code>	Records those changes, with a message

Step	Command or action	What it does
Push	<code>git push</code>	Sends the commits to GitHub

Table 1 lists the loop this homework practises. The steps are not a one-way sequence. Edit, render and commit form a small cycle you run many times over, and push happens whenever you want the work to be visible on GitHub.

i Note

Typed tables and computed tables are different things. A short overview like Table 1 is typed out in Markdown, which is what this week asks for. From week 3, a table built from data goes through `knitr::kable()` in a code chunk instead, and the label and caption move into the chunk options.

A first code chunk

This week does not require any R code, but the sneak peek from the session is worth repeating, because it is the difference between a text document and a Quarto document. A chunk is fenced with three backticks and `{r}`, and its output appears in the rendered PDF.

```
weeks_in_term <- 13
weeks_in_term * 90 / 60
```

```
[1] 19.5
```

The number above was not typed into the text. It was computed when this document was rendered, which is the property that makes the whole approach reproducible.

Exercise 5: reflection

The reflection below is one student's honest answer. There is no correct reflection, and a short specific one is worth more than a long general one.

The part I found hardest was understanding what staging is for. I expected `git commit` to save everything I had changed, and my first commit contained only the one file I had clicked the plus next to, which I did not notice until the history looked wrong on GitHub. Rendering to PDF also failed the first time with an error about LaTeX, and I could not tell whether the problem was my document or my installation. Running `quarto check` was what told me it was the installation. I am still unsure how often to commit: I understand that once at the end is too rarely, but I cannot yet judge whether every saved edit is too often.

That last sentence is exactly the kind of thing worth writing. It is specific, it is answerable, and it tells the instructor what to cover next.

The git history you should end up with

Three commits is the minimum, and each one should say what changed. A history like this can be read months later:

```
$ git log --oneline
9f2c1ab Reflect on the workflow, and render the final PDF
4be77d0 Add the Quarto logo and the workflow table, both cross-referenced
1d9e5c3 Describe Quarto, Git and GitHub in my own words
8a0f412 Set up the YAML header with title, author, date and toc
```

Compare that with `update`, `stuff` and `final`, which is a history recording only that something happened. A useful message says what changed and, where it is not obvious, why.

Two further points. The rendered PDF is committed as well as the source, because the homework asks for both. And if you used an AI tool to understand an error message or to check your reasoning, say so in the commit message or in the reflection. Declaring assistance is part of the workflow in this course, not an admission of anything.

Common misconceptions

! Important

The repository has to be created inside the course organisation. The GitHub page for creating a repository from a template defaults the Owner dropdown to your personal account, and a repository created there is invisible to the course tracking, so it does not count as submitted. Use the link on the schedule page, then check that the Owner dropdown reads `qmir-2026-fall` before you click the green button.

⚠ Warning

One commit at the end is not a history. A single commit holding the finished document practises none of what this homework is for, because the point is to record the work in steps another person can follow. Commit after each exercise, which gives you four or five commits at no extra cost.

⚠ Warning

A label without a caption is not a cross-reference. If you give an image a `#fig-` label but no caption text, Quarto has nothing to number and the rendered document shows a dangling `?fig-name` where your reference should be. The same applies to a table whose caption line is missing. Check the rendered PDF rather than the source, because this is a mistake you can only see in the output.

💡 Tip

Render early, render often, and treat a failed render as the test failing. Rendering is the only thing that proves the document still works, so a render after every small change turns a mysterious error at midnight into an obvious one you caused ten seconds ago. The habit costs nothing now and saves whole evenings once there are models in the document.

Session info and execution time

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24 ucrt)
```

Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: Europe/Berlin
tzcode source: internal

attached base packages:
[1] stats graphics grDevices utils datasets methods base

loaded via a namespace (and not attached):
[1] compiler_4.6.1 fastmap_1.2.0 cli_3.6.6 tools_4.6.1
[5] htmltools_0.5.9 otel_0.2.0 yaml_2.3.12 rmarkdown_2.32
[9] knitr_1.52 jsonlite_2.0.0 xfun_0.60 digest_0.6.39
[13] rlang_1.3.0 evaluate_1.0.5

```
end_time <- Sys.time()
cat(
  "Total execution time:",
  round(difftime(end_time, start_time, units = "mins"), 1),
  "minutes\n"
)
```

Total execution time: 0 minutes